Desafío II: Ude@Book

EMILIANO DE JESUS LINCE DIAZ
Facultad de Ing. Electrónica UdeA
Correo: dejesus.lince@udea.edu.co

ESTEBAN GARCES ATEHORTUA
Facultad de Ing. Electrónica UdeA
Correo: esteban.garces2@udea.edu.co

I. DESCRIPCIÓN DEL DESAFÍO Y RETOS PRINCIPALES

El desafío consiste en desarrollar Ude@Book, una red social en C++ que permita a los usuarios interactuar con sus amigos, consultar perfiles, compartir mensajes y recibir recomendaciones de nuevos contactos. El programa debe trabajar con una comunidad de al menos 1000 usuarios, cuyos datos se encuentran inicialmente almacenados en archivos. Al iniciar la aplicación, esta información debe cargarse en la memoria para que las consultas se realicen sin necesidad de leer los archivos constantemente.

El desarrollo presenta varias dificultades debido a las restricciones establecidas en el enunciado. No se permite utilizar contenedores de la STL ni la palabra reservada `struct`, tampoco se puede utilizar herencia y deben evitarse las funciones propias de C, como `printf` o `malloc`. Por esta razón, será necesario implementar nuestras propias estructuras de datos y prestar atención al manejo de la memoria dinámica.

A partir de estos requisitos, identificamos los siguientes retos principales:

- Manejo de las relaciones de amistad: El programa debe permitir consultar las amistades de cada usuario, aceptar solicitudes y encontrar personas que tengan amigos en común. El reto está en realizar estas consultas de manera eficiente, especialmente cuando se trabaja con una red de más de 1000 usuarios. Por ejemplo, una matriz de 1000×1000 bits necesita aproximadamente 125 KB para representar las relaciones, sin contar la memoria adicional que requiere el programa.
- Control del consumo de memoria y las iteraciones: Al finalizar cada funcionalidad, se debe mostrar cuánta memoria está utilizando el programa y cuántas iteraciones se realizaron. Esta medición debe tener en cuenta tanto la memoria dinámica (*heap*) como las variables locales y los parámetros que se pasan por valor (*stack*). Además, se deben contar las iteraciones realizadas directamente en una función y las que se producen dentro de otras funciones o estructuras utilizadas durante su ejecución.
- Carga y actualización de los datos: La información de los usuarios, las amistades, las solicitudes, los álbumes, las fotografías y los mensajes debe cargarse en la memoria al iniciar el programa. Durante la ejecución, los archivos no deben utilizarse para realizar consultas o intercambiar información. Al cerrar el programa, se deben guardar únicamente los cambios realizados, incluyendo las solicitudes aceptadas, los mensajes nuevos, el estado de lectura de los mensajes y los *likes* de las fotografías.
- Implementación de estructuras de datos propias: Las restricciones del proyecto obligan a crear clases que permitan manejar listas dinámicas y cadenas de caracteres sin depender de los contenedores de la STL. Estas clases deben gestionar correctamente la memoria y permitir copiar objetos cuando sea necesario, evitando pérdidas de memoria (*memory leaks*) y copias innecesarias que puedan afectar el rendimiento.
- Recomendaciones de amistad y selección de fotografías: El sistema debe recomendar contactos teniendo en cuenta la cantidad de amigos en común y mostrar los resultados en orden de mayor a menor coincidencia. Asimismo, al ingresar, se deben seleccionar tres fotografías de perfil de amigos, utilizando las probabilidades establecidas en el enunciado: 1, $2{,}5$ y $3{,}5$. La selección no debe repetir fotografías y solo se permite actualizar el grupo una vez.
- Consulta de perfiles y álbumes: Al visitar el perfil de un amigo, se deben mostrar los álbumes que sean públicos o estén disponibles para sus amigos. La navegación entre las fotografías debe funcionar como un carrusel circular, de modo que, al llegar a la última foto, sea posible volver a la primera y viceversa. También se debe permitir dar y quitar *likes*.
- Gestión de mensajes privados: Los mensajes solo se pueden enviar a personas que sean amigas del usuario y no pueden estar vacíos ni superar los 512 caracteres. Además, deben organizarse por fecha para facilitar la consulta de mensajes nuevos y la búsqueda dentro de un rango de fechas, incluyendo ambos extremos. Cuando un

mensaje nuevo sea leído, su estado debe actualizarse y conservarse al cerrar el programa.

II. DISEÑO MODULAR

II-A. Criterio de división en módulos

Para desarrollar el proyecto, se decidió dividir el programa en varios módulos, cada uno encargado de una parte específica del sistema. La idea es que el código sea fácil de entender, modificar y probar, sin que un cambio en una parte afecte innecesariamente a las demás. Además, esta organización permite cumplir con el requisito de trabajar con varios archivos de C++, utilizando archivos de cabecera (.h) y archivos de implementación (.cpp) como se exige en la guía.

II-B. Dependencias entre los módulos

La organización busca que los módulos tengan responsabilidades claras y que las dependencias entre ellos sean fáciles de identificar. De esta manera, las clases básicas sirven de apoyo para las entidades y estructuras, mientras que `RedSocial` utiliza estos componentes para ejecutar las funciones del programa.

- Coordinación del sistema: La clase `RedSocial` se encarga de reunir las diferentes partes del programa. Desde ella se gestionan las operaciones relacionadas con los usuarios, las amistades, los mensajes y las demás funcionalidades. Esto permite que las reglas principales del sistema no queden repartidas sin control entre varias clases.
- Separación del menú: La clase `Menu` se encarga unicamente de mostrar las opciones y recibir las entradas del usuario.

II-C. Decisiones de diseño

Para mantener el código organizado, las validaciones se realizarán en las clases que tengan la información necesaria para comprobar cada condición.

- Validaciones de los mensajes: La clase `Mensaje` se encargará de controlar que el texto no esté vacío y que no supere los 512 caracteres. De esta manera, se evita que se creen mensajes que incumplan estas condiciones.
- Validaciones de las solicitudes de amistad: La clase `RedSocial` comprobará si dos usuarios ya son amigos o si existe una solicitud pendiente antes de permitir el envío de otra. Esta validación necesita consultar las relaciones entre los usuarios y no puede depender únicamente de los datos de una persona.
- Separación del almacenamiento: La lectura de los datos iniciales y la escritura de los cambios se manejarán desde el módulo `Persistencia`. Así, las clases encargadas de las funcionalidades no tendrán que trabajar directamente con los archivos, lo que facilita su desarrollo y mantenimiento.

II-D. Flujo de ejecución de la aplicación

El funcionamiento general del programa se divide en las siguientes etapas:

1. Inicio del programa: En `main.cpp` se inicia la aplicación y se prepara el objeto principal `RedSocial`.
2. Carga de información: Antes de permitir la interacción, se leen los archivos con los datos de los usuarios, las amistades, las solicitudes, los álbumes, las fotografías y los mensajes. Esta información se organiza en memoria para que pueda consultarse durante la ejecución sin recurrir nuevamente a los archivos.
3. Interacción con el usuario: Una vez cargados los datos, se muestra el menú por consola. El usuario puede seleccionar las diferentes opciones disponibles, como ingresar a su cuenta, gestionar amistades, consultar perfiles o revisar mensajes.
4. Ejecución y medición: Cada opción ejecuta las operaciones utilizando los datos que están en memoria. Al terminar una funcionalidad, se deben mostrar las iteraciones realizadas y el consumo de memoria, incluyendo la memoria dinámica, las variables locales y los parámetros que se pasan por valor.
5. Cierre del programa: Cuando el usuario decide salir, se guardan en los archivos los cambios realizados durante la ejecución. Entre ellos se encuentran las solicitudes aceptadas, los mensajes nuevos, los estados de lectura y los likes. Finalmente, se libera la memoria dinámica utilizada para evitar pérdidas de memoria.

II-E. Mapa de módulos y responsabilidades

La división propuesta para el programa, donde se detallan las capas, los módulos y sus respectivas responsabilidades, se encuentra documentada en la Tabla II, ubicada en la Sección V al final de este documento.

III. SELECCIÓN DE ESTRUCTURAS DE DATOS Y VERIFICACIÓN DE EFICIENCIA

Para el desarrollo del sistema se seleccionarán las estructuras de datos teniendo en cuenta la cantidad de usuarios esperada y las operaciones principales de la aplicación. Se buscará reducir los tiempos de búsqueda y procesamiento, evitando al mismo tiempo un consumo innecesario de memoria. A continuación, se describirán las estructuras propuestas para cada funcionalidad, sus costos estimados para una red de 1000 usuarios y las alternativas que se descartarán.

- *Búsqueda de usuarios por nombre:*
  Para esta funcionalidad se implementará una tabla hash con encadenamiento para manejar las colisiones. La función hash calculará la posición correspondiente a partir del nombre del usuario, lo que permitirá acceder directamente a la posición donde se almacenará la información. Para una red de 1000 usuarios, la búsqueda tendrá una complejidad promedio de $O(1)$, siempre que los elementos se distribuyan adecuadamente entre las posiciones de la tabla. En condiciones favorables, se espera que la búsqueda requiera pocas comparaciones. Sin embargo, en el peor caso, si se presentan muchas colisiones, la complejidad podrá llegar a $O(N)$.
  Como alternativa se considerará un arreglo ordenado con búsqueda binaria, cuya complejidad será de $O(\log N)$. Para 1000 usuarios, se necesitarán aproximadamente 10 comparaciones en el peor caso. Esta alternativa se descartará porque la tabla hash ofrecerá un mejor tiempo promedio de búsqueda cuando la función de dispersión y el manejo de colisiones estén bien implementados.
- *Amigos en común y sugerencias de amistad (Top-K):*
  Las relaciones de amistad se representarán mediante una matriz de adyacencia almacenada a nivel de bits. Para identificar candidatos a sugerencias, se combinarán mediante operaciones lógicas `OR` las filas correspondientes a los amigos directos del usuario. Posteriormente, se excluirán los usuarios que ya sean amigos y el propio usuario.
  Para calcular los amigos en común entre dos usuarios, se realizarán operaciones `AND` entre las palabras de 64 bits que representarán sus conexiones. Con 1000 usuarios, cada fila requerirá aproximadamente 16 palabras de 64 bits. Por tanto, una intersección completa entre dos filas necesitará alrededor de 16 operaciones `AND`, además de las operaciones necesarias para contar los bits activos.
  Para las sugerencias Top-K se utilizará un arreglo temporal de tamaño K, por ejemplo, de cinco posiciones. Cada candidato se insertará según su puntuación, manteniendo únicamente las mejores sugerencias. El costo de esta inserción será de hasta $O(K)$ por candidato, mientras que el costo total dependerá de la cantidad de candidatos evaluados y de las operaciones necesarias para calcular sus puntuaciones.
  Como alternativa se considerará almacenar las amistades en arreglos de identificadores y calcular las intersecciones mediante recorridos lineales. Esta opción se descartará porque podría requerir más comparaciones para conjuntos grandes. También se evitará ordenar la totalidad de los usuarios, ya que mantener únicamente los primeros K candidatos permitirá reducir el trabajo cuando la cantidad de sugerencias sea considerable.
- *Consulta de mensajes por rango de fechas:*
  Los mensajes de cada conversación se almacenarán en una instancia independiente de la clase `ListaDinamica`. Como los mensajes se incorporarán en orden cronológico, se mantendrá el historial ordenado para facilitar las consultas posteriores.
  Para una conversación con M mensajes, se utilizará búsqueda binaria para localizar el primer mensaje que cumpla con la fecha inicial del intervalo. Esta operación tendrá una complejidad de $O(\log M)$. A partir de esa posición, se recorrerán los mensajes hasta superar la fecha final solicitada, por lo que el costo total de la consulta será $O(\log M+R)$, donde R representará la cantidad de mensajes examinados dentro del intervalo.
  Como alternativa se considerará una lista enlazada con recorrido secuencial, cuya búsqueda por fecha podrá requerir hasta $O(M)$ operaciones. Esta alternativa se descartará porque no permitirá acceder directamente a una posición intermedia, mientras que la búsqueda binaria aprovechará el acceso por índice de la estructura dinámica.
- *Selección ponderada de fotos de perfil:*
  Para realizar la selección ponderada se asignarán pesos enteros según la cantidad de amigos en común de cada candidato: 2 puntos para 10 amigos en común o menos, 5 puntos entre 11 y 20, y 7 puntos para más de 20.
  El algoritmo calculará el peso de cada candidato, sumará todos los pesos y generará un número aleatorio entre cero y la suma total. Después, recorrerá los candidatos y restará sus pesos hasta identificar el seleccionado. De esta manera, los candidatos con mayor peso tendrán una probabilidad más alta de ser elegidos. Una vez seleccionado un candidato, se excluirá de las siguientes rondas para evitar repeticiones.
  Si se evalúan C candidatos, la asignación de pesos y la selección tendrán un costo de $O(C)$, sin contar el cálculo previo de los amigos en común. Si este último cálculo requiere comparar filas completas de una matriz de 1000 usuarios, se necesitarán aproximadamente 16 operaciones `AND` por pareja evaluada. El costo total dependerá de la cantidad de candidatos y de la estrategia utilizada para obtener sus puntuaciones.
  Como alternativa se considerará ordenar todos

los candidatos según su puntuación y seleccionar uno de ellos posteriormente. Esta opción se descartará porque un ordenamiento completo podrá requerir $O(C \log C)$ operaciones, mientras que el método ponderado permitirá realizar la selección en tiempo lineal respecto a la cantidad de candidatos.

- *Carrusel circular de fotos:*
  Los álbumes se recorrerán de forma secuencial y las fotos de cada álbum se visualizarán mediante un carrusel circular implementado sobre un arreglo dinámico. Si el índice actual será i y el álbum contendrá F fotos, el avance se calculará mediante

$$i_{\text{sig}} = (i + 1) \text{ mód } F$$

  mientras que el retroceso se calculará mediante

$$i_{\text{ant}} = (i - 1 + F) \text{ mód } F.$$

  Estas operaciones permitirán pasar de la última foto a la primera y viceversa. Tanto el avance como el retroceso tendrán una complejidad de $O(1)$, independientemente de que el álbum contenga pocas o muchas fotografías. Para evitar errores, se verificará que el álbum tenga al menos una foto antes de aplicar la operación modular.
  Como alternativa se considerará una lista doblemente enlazada, que también permitirá recorrer las fotos en ambos sentidos. Sin embargo, se descartará porque requerirá almacenar punteros adicionales en cada nodo y no ofrecerá una ventaja significativa para esta funcionalidad, ya que el arreglo permitirá acceder directamente a cualquier foto mediante su índice.

IV. PROBLEMAS DE DESARROLLO

- Errores en la medición de la memoria, en este caso siempre nos generaba un mismo valor de memoria y no cambiaba, y esto se daba porque solo recorriamos el tamaño de esta y no la capacidad.
- Teníamos las variables locales fuera del reporte de memoria, las listas temporales no existian cuando el reporte se generaba.
- En la tabla Hash se rechazaban algunos usuarios luego de cierta capacidad, para esto aplicamos un rehash.

V. ANEXOS: TABLAS GENERALES

Tabla I
EFICIENCIA ESTIMADA DE LAS ESTRUCTURAS DE DATOS PARA 1000 USUARIOS

Funcionalidad	**Estructura elegida**	**Costo estimado**	**Alternativa descartada**
Búsqueda de usuarios	Tabla hash con encadenamiento	$O(1)$ promedio, con una distribución adecuada; hasta $O(N)$ en el peor caso	Arreglo ordenado con búsqueda binaria, $O(\log N)$
Amigos en común y sugerencias	Matriz de adyacencia a nivel de bits y arreglo Top-K	Aproximadamente 16 operaciones `AND` por intersección de dos filas; inserción de hasta $O(K)$ por candidato	Arreglos de identificadores con intersección lineal y ordenamiento completo
Mensajes por rango de fechas	`ListaDinamica` y búsqueda binaria	$O(\log M + R)$, donde M es el total de mensajes y R los mensajes examinados	Lista enlazada con recorrido secuencial, $O(M)$
Selección ponderada	Arreglo de candidatos y pesos enteros	$O(C)$ para la selección, sin incluir el cálculo previo de las puntuaciones	Ordenamiento completo, $O(C \log C)$
Carrusel de fotos	Arreglo dinámico y aritmética modular	$O(1)$ por movimiento	Lista doblemente enlazada con punteros adicionales

Tabla II
MAPA DE MÓDULOS Y RESPONSABILIDADES

Capa	Clases o módulos	Responsabilidad
Base	`Fecha`, `Cadena`, `ListaDinamica`	Contienen las herramientas básicas para manejar fechas, cadenas de caracteres y listas dinámicas. Se busca que puedan probarse por separado antes de integrarlas al resto del programa.
Entidades	`Foto`, `Album`, `Mensaje`, `Usuario`	Representan la información de los usuarios y los elementos que utiliza la red social, como las fotografías, los álbumes y los mensajes. También permiten controlar los datos propios de cada objeto.
Estructuras de apoyo	`RelacionesAmistad`, `TablaHashUsuarios`	Permiten organizar las relaciones entre usuarios y facilitar la búsqueda de perfiles y el cálculo de amigos en común.
Servicios	`Persistencia`, `MedidorRecursos`	Se encargan de guardar los cambios en los archivos y de calcular la memoria utilizada y las iteraciones realizadas durante las operaciones. `Persistencia` se utilizará como módulo del código, pero no se incluirá en el diagrama de clases UML.
Coordinación	`RedSocial`	Se encarga de conectar los diferentes componentes y ejecutar las operaciones principales de la red social.
Presentación	`Menu`, `main.cpp`	Permiten interactuar con el usuario mediante la consola, mostrar las opciones disponibles y poner en marcha el programa.